Topology Statistics Export Specification

Normative TS5 API for Stable JSON and CSV Output in mdstats

mdstats

2026-07-14 (implemented TS5 revision)

Contents

Purpose and status

This document specifies the implemented module

`mdstats/io/topology_statistics.py`

introduced in `mdstats 0.17.0a5` as the machine-readable output half of TS5.

The module exports completed TS1, TS2, or TS4 result objects. It does not read source trajectories, rebuild graph statistics, or define a second statistical schema.

result payloads are authoritative; exports are deterministic views .

Public schema

```
CANONICAL_TOPOLOGY_STATISTICS_EXPORT_SCHEMA =
    "mdstats.topology-statistics.export.v1"
```

The JSON file uses the result object's own canonical `to_dict()` schema and digest. The TS5 export schema identifies the manifest and table convention, not a competing replacement for TS0–TS4 serialization.

Public data structures

`TopologyStatisticsTable`

```
@dataclass(frozen=True, slots=True)
class TopologyStatisticsTable:
    name: str
```

```
    columns: tuple[str, ...]
    rows: tuple[tuple[str | int | float | bool | None, ...], ...]

    @property
    def n_rows(self) -> int: ...

    def write_csv(
        self,
        path: str | Path,
        *,
        overwrite: bool = False,
    ) -> Path: ...
```

Constraints:

- `name` is nonempty;
- columns are nonempty and unique;
- every row has exactly one value per column;
- CSV is UTF-8 with `\n` line endings;
- existing paths are not overwritten unless requested.

`TopologyStatisticsExportManifest`

```
@dataclass(frozen=True, slots=True)
class TopologyStatisticsExportManifest:
    output_directory: Path
    json_path: Path | None
    csv_paths: tuple[Path, ...]
    table_row_counts: Mapping[str, int]
    canonical_schema_version: str
```

The manifest records produced files; it is not itself a scientific result.

Public functions

Table construction

```
def build_topology_statistics_tables(
    result: AtomicConnectivityStatistics
          | FrameworkTopologyStatistics
          | TopologyStatistics,
) -> tuple[TopologyStatisticsTable, ...]:
    ...
```

The returned table names and columns are deterministic. Tables are long-form so that species pairs, descriptors, bridge signatures, and catalog sizes can vary without changing column count.

JSON writer

```
def write_topology_statistics_json(
    result,
    path: str | Path,
    *,
    overwrite: bool = False,
    indent: int = 2,
) -> Path:
    ...
```

The writer emits `result.to_dict()` with sorted keys. Restoring the correct result class from this payload must preserve its digest.

Combined export

```
def export_topology_statistics(
    result,
    output_directory: str | Path,
    *,
    prefix: str = "topology_statistics",
    write_json: bool = True,
    write_csv: bool = True,
    overwrite: bool = False,
) -> TopologyStatisticsExportManifest:
    ...
```

At least one output format should normally be enabled. The function creates the output directory if required.

Initial stable table set

Shared table

`frame_axis` contains:

```
result_position
collection_frame_index
frame_id
step
time
time_unit
```

Missing step or time values are empty CSV cells.

Atomic tables

```
atomic_catalog_occupancy
atomic_state_assignments
atomic_total_edge_series
atomic_pair_count_series
atomic_pair_count_distribution
atomic_contact_occupancy
atomic_state_residence            # when TS3 is present
atomic_state_transitions          # when TS3 is present
```

For an atomic species pair $A - B$, the distribution table stores exact rows

$$(A - B, n, \nu_n, p_n), \qquad p_n = \frac{\nu_n}{F}.$$

Contact occupancy uses gauge-invariant canonical atom pairs, not state-local periodic image labels.

Framework tables

```
framework_catalog_occupancy
framework_state_assignments
framework_descriptor_series
framework_descriptor_distribution
framework_endpoint_pair_series
framework_endpoint_pair_distribution
framework_bridge_signature_series
framework_bridge_signature_distribution
framework_edge_occupancy
framework_state_residence          # when TS3 is present
framework_state_transitions        # when TS3 is present
```

Canonical framework edge keys are encoded as sorted compact JSON inside one CSV cell so whole-path identity, periodic translation, and linker order remain lossless.

Combined tables

A `TopologyStatistics` export includes both branches plus:

```
cross_layer_assignments
cross_layer_contingency
cross_layer_boundaries             # when boundary statistics are present
```

The contingency table stores one row per atomic-state/framework-class pair:

$$(a, k, C_{ak}, C_{ak}/F).$$

Naming contract

For prefix `case`, files are named:

```
case.json
case_<table_name>.csv
```

Table names use lowercase snake case. Scientific terms are explicit: `state`, `class`, `contact`, `framework_edge`, and `bridge_signature` are not substituted for one another.

Input and output constraints

- Inputs must be validated TS1, TS2, or TS4 results.
- Paths must be writable filesystem paths.
- Prefixes must be nonempty.
- Export does not infer missing temporal statistics.
- Empty optional tables are allowed and retain their headers.
- JSON uses the branch result's own canonical schema and digest.
- CSV values are scalar; structured edge keys are canonical JSON strings.

Algorithm and scaling

Table construction traverses completed arrays and records only. If the export contains F frames, P species-pair series, D framework descriptors, and U resolved contacts or framework edges, the dominant cost is

$$O(F(P + D) + U).$$

The exporter does not expand source graph catalogs beyond statistics already stored in the result. JSON size is determined by the authoritative result payload; CSV size is determined by the selected stable tables.

Edge cases and warnings

- Exporting all per-frame pair series can create large CSV files for many species pairs or very long trajectories.
- CSV preserves exact scalar values but does not preserve Python types beyond standard text parsing.
- An empty transition table means no transitions or unavailable temporal output; the table name and result metadata distinguish the context.
- Existing files are protected by default.
- CSV output is intended for tables, not as a substitute for the canonical JSON payload.
- Framework edge JSON cells should be parsed as JSON before structural comparison.

Validation requirements

Tests must verify:

- deterministic table names and column order;
- exact PMF frequencies and probabilities;
- JSON round-trip through the source result class;
- no duplicate table names;
- correct atomic-only, framework-only, and combined table sets;
- overwrite protection and explicit overwrite behavior;

- UTF-8 CSV readability with standard-library `csv`;
- zero mutation of source result objects.